PRÁTICAS EM SISTEMAS OPERACIONAIS

AULA 5

Prof. André Luiz de Souza Paula

CONVERSA INICIAL

Sistemas Operacionais GNU/LINUX

Objetivo: compreender os comandos de gerenciamento de processos no ambiente GNU/Linux. Assim como monitorar, controlar os processos em execução e administrar usuários e grupos no ambiente GNU/Linux.

Temas de estudo:

1. Níveis de execução
2. Serviços e Daemon
3. Gerenciamento de processos
4. Administração de grupos e usuários
5. Política de senhas

TEMA 1 – NÍVEIS DE EXECUÇÃO

Ao ligarmos um computador, a BIOS executa uma série de operações antes de iniciar o boot do sistema operacional, conforme sequência a seguir:

a) Computador é ligado pelo interruptor, iniciando a energização dos circuitos elétricos;
b) O POST (*Power On Self Test*) executa várias operações;
c) O BIOS (*Basic Input Output System*) lê o setor de boot (*MBR*);
d) Executa um programa chamado gerenciador de inicialização;
e) É feito o carregamento do kernel em um determinado nível de execução;
f) O kernel executa o *init* (PID=1. Isso mesmo, o *init* é uma aplicação);
g) O *init* lê o arquivo /etc/inittab;
h) O *init* executa os scripts de acordo com o nível de execução selecionado;
i) Sistema pronto para uso.

Runlevels, ou níveis de execução, constituem uma maneira de controlar como a máquina será utilizada. Cada *runlevel* é configurado para que um determinado número de processos específicos esteja em execução. Convencionalmente temos 7 (sete) níveis de *runlevels* do 0 ao 6, com diferentes características de execução do Sistema Operacional.

Apesar desta seção tratar de sistemas GNU/Linux, deve-se observar que executar o kernel de um sistema operacional com diferentes níveis de execução

é uma característica de vários sistemas operacionais. O MS-Windows, por exemplo, é capaz de iniciar com “Modo de Segurança”, ou seja, monousuário ou “Modo Normal” (multiusuário com rede).

1.1 Gerenciador de inicialização

O gerenciador de inicialização (partida) é o responsável por carregar o sistema operacional durante o processo de inicialização, além de, nos casos em que houver mais de um sistema operacional, permitir que seja feita a escolha de qual sistema operacional deverá ser carregado. Este programa geralmente fica gravado no setor de boot de uma partição ativa ou na *Master Boot Record* (MBR) do disco.

Exemplo resumido de uma configuração para carregamento do Kernel, utilizando GRUB2 e Debian GNU/Linux (*/boot/grub.cfg*):

```
menuentry 'Debian GNU/Linux com Kernel x86_64-3.12' --class gnu-linux --class gnu --class os $menuentry_id_option 'gnulinux-simple-024579f5-42ad-4bd1-8f0f-23da56a4c896' {
    load_video
    insmod gzio
    insmod part_msdos
    insmod xfs
    set root='hd0,msdos5'
    linux   /boot/kernel-genkernel-x86_64-3.12 root=UUID=024579f5-42ad-4bd1-8f0f-23da56a4c896 ro splash=silent, console=tty1 quiet nomodeset resume=swap:UUID=42d28857-66f7-4dc4-9f5a-b0acc6d93d65 real_resume=UUID=42d28857-66f7-4dc4-9f5a-b0acc6d93d65 root=UUID=024579f5-42ad-4bd1-8f0f-23da56a4c896
    initrd  /boot/initramfs-genkernel-x86_64-3.12
}
```

Sobre o quadro exposto, vamos analisar alguns parâmetros:

menuentry: descrição que o usuário verá ao inicializar o seu computador;

load_video / insmod {gzio, part_msdos, xfs}: são módulos adicionais que serão carregados para auxiliar o GRUB a ler o disco;

Linux: essa é a principal opção que indica qual é a imagem do kernel a ser utilizada, sua localização e opções adicionais;

Initrd: *Initial RamDisk* é um arquivo que contém um sistema de arquivos a ser carregado em memória, ele será utilizado pelo kernel durante a inicialização;

Existem outras variáveis importantes em /etc/default/grub que afetam o comportamento do gerenciador de inicialização:

GRUB_DEFAULT: sistema operacional padrão a ser inicializado, por ordem de configuração;

GRUB_TIMEOUT: tempo de espera pela interação do usuário antes de entrar no *GRUB_DEFAUT*;

GRUB_CMDLINE_LINUX_DEFAULT: opções de inicialização do Kernel Linux.

O GRUB (sigla de *GRand Unifield Bootloader*) é o gerenciador de partida mais utilizado pelas distribuições GNU/Linux atualmente. Trata-se de um sistema *multiboot* muito versátil e que tem suporte para diversos sistemas operacionais. Entretanto, existem vários outros, o mais clássico deles é o *Linux Loader (LILO)*, que foi utilizado por muitos anos e sistemas clássicos ainda utilizam. Para finalizar esta seção, vamos apresentar uma imagem do GRUB no momento da sua execução e um quadro comparativo entre os diversos gerenciadores de inicialização:



Exemplos de sistemas de inicialização (*Loader*) e os sistemas operacionais suportados:

NOME	Sistemas Operacionais suportados				
	Windows 9x/Me	**Windows Vista/7/8/10**	**Linux**	***BSD**	**Mac OS X**
Acronis OS Selector	Sim	Sim	Sim	Sim	Sim
BootX (Apple)	Não	Não	Não	Não	Sim
BootX (Linux)	Não	Não	Sim	Não	Não
GRUB Legacy e GRUB4DOS	Sim	Carrega o Windows Boot Manager	Sim	Carrega o bootloader	Sim
GNU GRUB	Sim	Carrega o Windows Boot Manager	Sim	Sim	Sim
Grub2Win	Sim	Carrega o Windows Boot Manager	Sim	Sim (FreeBSD)	Carrega o Clover
Gujin	Sim	Sim	Sim	Não	Não
systemd-boot / Gummiboot	Não	Carrega o Windows Boot Manager	Sim	UEFI somente	Sim
iBoot	Não	Não	Não	Não	Sim
LILO	Sim	Sim	Sim	Carrega o biosboot (FreeBSD, PC-BSD, ...)	Não
MasterBooter	Sim	Sim	Sim	Sim	Não
NTLDR	Sim	Não	Carrega o GRUB4DOS	Não	Carrega o Darwin bootloader
SYSLINUX	Sim	Carrega o Windows Boot Manager	Sim	via mboot.c32 module [1]	Não
Windows Boot Manager	Sim	Sim	Carrega o GRUB ou LILO	Não	Não

1.2 Níveis de execução (runlevel)

A tabela a seguir mostra os níveis de execução de acordo com a especificação LSB (*Linux Standard Base*) e seus significados:

Runlevel	Significado
0	Desligar o sistema
1	Modo de usuário único (modo de manutenção)
2	Modo multiusuário sem rede
3	Modo multiusuário com rede
4	Não usado
5	Modo multiusuário completo com GUI (*runlevel* 3 com interface gráfica)
6	Reiniciar o sistema

A utilização dos níveis de execução pode variar em cada distribuição GNU/Linux, por isso vamos analisar os níveis de execução configurados pelo Debian GNU/Linux e seus derivados:

O nível zero (0) é utilizado para realizar o desligamento (*shutdown*) normal da máquina;

O nível 1 (modo mono usuário) é o modo no qual o Kernel do Linux só carrega os recursos necessários para o funcionamento básico do Sistema Operacional e disponibiliza apenas acesso ao superusuário do sistema (root), de modo geral, para que este possa realizar a manutenção de alguma falha do sistema;

Os níveis 2 a 5 são utilizados para carga dos diferentes serviços providos pelo sistema, e possuem uma característica comum entre si: atuarem no modo multiusuário;

O nível 6 é usado para reinicializar (*reboot*) o sistema.

1.3 Identificando o *runlevel* em utilização

Temos basicamente duas formas de identificar o *runlevel* carregado:

1. Uma delas é por meio do comando: `~# ps aux | grep init`

   A saída deste comando será algo como:

```
root  1  0.0  0.0  1984  688 ? Ss 07:45 0:01 init [2]
```

Neste exemplo, o nível de execução atual do Sistema Operacioal é o 2, representado pela indicação “init [2]” presente na primeira linha da saída do comando;

2. Outra forma de se identificar o runlevel carregado é por meio do comando "runlevel", por exemplo:

```
~# runlevel
```

A saída deste comando será algo como:

```
N 5
```

O "N 5" da saída do comando "runlevel" indica que o nível de execução atual do sistema é o 5, se você mudar o nível para outro qualquer, o nível anterior será mostrado no lugar do "N" exibido na saída acima. Desta forma é possível verificar qual foi o último e o corrente.

TEMA 2 – SERVIÇOS E DAEMON

Os serviços que iniciam com um nível de execução são alocados pelos sistemas *systemv* (legado) ou *systemd*. O **systemd** é um subsistema de inicialização dos sistemas operacionais GNU/Linux. Ele controla os serviços que são executados em segundo plano (*daemons*), auxiliando a instalação de novos serviços, manutenção dos serviços existentes e exclusão de serviços que não são mais necessários. Esses programas executados em segundo plano são chamados de *daemons* e iniciados pelo processo principal do *systemd* (*init)*.

2.1 Verificar o status de um serviço no systemd

Sintaxe:

```
~# systemctl status <DAEMON>.service
```

Exemplo:

```
~# systemctl status cron.service

cron.service – Command Scheduler
Loaded: loaded (/lib/systemd/system/cron.service; enabled)
Active: active (running) since Mon, 28 Nov 2023 18:09:57 -0300;
7min ago
Main PID: 673 (cron)
CGroup: name=systemd:/system/cron.service
+ 673 /usr/sbin/cron -n
```

2.2 Parar um serviço no systemd

Sintaxe:

```
~# systemctl stop <DAEMON>.service
```

Exemplo:

```
~# systemctl stop cron.service
```

2.3 Para iniciar um serviço no systemd

Sintaxe:

```
~# systemctl start <DAEMON>.service
```

Exemplo:

```
~# systemctl start cron.service
```

2.4 Para reiniciar um serviço no systemd

Sintaxe:

```
~# systemctl restart <DAEMON>.service
```

Exemplo:

```
~# systemctl restart cron.service
```

2.5 Para habilitar um serviço durante a carga do sistema no systemd

Sintaxe:

```
~# systemctl enable <DAEMON>.service
```

Exemplo:

```
~# systemctl enable cron.service
```

2.6 Para desabilitar um serviço durante a carga do sistema no systemd

Sintaxe:

```
~# systemctl disable <DAEMON>.service
```

Exemplo:

```
~# systemctl disable cron.service
```

2.7. Para mudar o *runlevel* para o modo texto (runlevel 3)

Sintaxe:

```
~# systemctl status <DAEMON>.service
```

Exemplo:

```
~# systemctl isolate multi-user.target
```

2.8. Para mudar o *runlevel* para o modo gráfico (runlevel 5)

Sintaxe:

```
~# systemctl status <DAEMON>.service
```

Exemplo:

```
~# systemctl isolate graphical.target
```

2.9. Listar todos os serviços (daemons) registrados

Sintaxe:

```
~# systemctl list-units
```

2.10 Dicas de utilitários

sysv-rc-conf

O sysv-rc-conf é outra ferramenta de configuração de nível de execução para sistemas baseados em Debian.

Para instalar e iniciar sysv-rc-conf no Ubuntu ou Debian:

```
~# apt install sysv-rc-conf
~# sysv-rc-conf
```

rcconf

O rcconf é uma interface mais simples de configuração do sistema de inicialização e serviços.

Para instalar e iniciar rcconf no Ubuntu ou Debian:

```
~# apt install rcconf
~# rcconf
```

Um processo nada mais é do que um programa que foi iniciado pelo usuário, ou sistema operacional, que recebe uma identificação (*Process ID - PID*). Este processo ficará no estado de pronto, aguardando execução, e terá sua execução controlada pelo Sistema Operacional. Vamos abordar diversos comandos que têm a finalidade de identificar, controlar e alterar parâmetros de processos.

Nem todo processo é facilmente identificado por um usuário, pois nem sempre está rodando em primeiro plano (em alguma janela ou terminal), muitos processos executam serviços do sistema e são executados em segundo plano (*background*). Também iremos abordar como tratar esse tipo de processo.

3.1 Comandos para manipulação de processos

1) Comando PS

O comando PS tem a finalidade de identificar os processos em execução e retorna na tela os processos em execução no sistema, ou usuário:

```
~# ps aux
USER  PID %CPU %MEM VSZ  RSS TTY STAT START    TIME COMMAND
root   1    0.0 0.0  63  668 ?   Ss Feb14 11:46 /sbin/init
root   2    0.0 0.0  0     0   ?   S      Feb14 0:04 [kthreadd]
root   3    0.0 0.0  0     0   ?   I<     Feb14 0:00 [rcu_gp]
root   4    0.0 0.0  0    0    ?   I<     Feb14 0:00 [rcu_par_gp]
aluno  5    0.0 0.0  0     0  ?   I<     Feb14 0:00 /bin/passwd
```

É possível exibir os processos por usuário: **`~# ps -u aluno`**

O comando **ps** oferece muitas opções, entretanto as mais usadas são:

a Exibe na tela os processos em execução por todos os usuários;

-a Exibe na tela os processos vinculados diretamente a um terminal, sem os processos de sessão;

-e, **-A** Exibe na tela todos os processos em execução;

-u Exibe os processos com a informação do nome dos usuários donos;

-x Exibe na tela a todos de processos, inclusive aqueles que não têm um terminal ligado a ele. Esses processos normalmente são processos servidores (*daemons*) que foram iniciados automaticamente pelo sistema operacional;

-f Exibe na tela os relacionamento entre os processos e seus processos filhos, em forma e árvore;

-H Exibe na tela o relacionamento dos processos em forma de árvore;

2) Comando TOP

O comando TOP tem a finalidade de identificar os processos em execução, a diferença com o PS é a interatividade (a ordem dos processos é por uso de processamento - CPU):

```
# top
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si
MiB Mem :  1956.2 total, 1407.5 free, 175.9 used, 372.8 buff/cache
MiB Swap:  0.0 total,      0.0 free,      0.0 used.   1620.9 avail Mem
PID USER PR  NI    VIRT    RES    SHR S  %CPU  %MEM   TIME+ COMMAND
1 root   20   0  167416  12784   8304 S   0.0   0.6  0:03.89 systemd
2 aluno  20   0       0      0      0 S   0.0   0.0   0:00.00 firefox
3 root    0   0       0      0      0 I   0.0   0.0   0:00.00 top
4 aluno   0   0       0      0      0 I   0.0   0.0   0:00.00 gedit
```

O comando **top** oferece muitas opções, entretanto as mais usadas são:

-b: inicia em modo batch, com objetivo de enviar a saída para outro programa ou arquivo, este modo não aceita interações;

-d n: especifica o atraso entre as atualizações de tela (*refresh*), que por padrão ocorre a cada 5 segundos. Sempre é bom lembrar que diminuir esse tempo pode afetar o desempenho do sistema;

-n num: o objetivo desta opção é fazer com o que o comando *top* não seja executado indefinidamente, ocorrendo um determinado número (*-n num*) de atualizações (*refresh*) é imediatamente encerrado;

-q: altera o comportamento do comando top para exibir as informações em tempo real, fazendo com que as atualizações (*refresh*) ocorram no menor tempo possível – isso impacta diretamente o desempenho do sistema;

-u: filtra os processos a serem exibidos, de acordo um usuário específico, informado nesta opção;

-p: filtra os processos a serem monitorados, sendo necessário informar um ou mais identificadores de processos (*PID*).

O comando ***top*** ainda suporta interação com o usuário alterando o seu comportamento durante a sua execução. Os comandos de interação serão realizados pelas teclas listadas a seguir:

- **Z**: altera a aparência, fazendo com que o esquema de cores se altere de acordo com a necessidade do ambiente;
- **F**: altera as colunas que estão sendo exibidas, podendo adicionar novas colunas com mais informações;
- **u**: filtra, em tempo real, exibindo processos de um determinado usuário;
- **k**: envia um sinal para um determinado processo, utilizando os sinais previstos no comando *kill (*verifique *~# kill -l);*
- **r**: executa o comando *renice* em um PID específico;
- **R**: altera a visão de uso de CPU em ordem crescente ou decrescente;
- **q**: interrompe a execução do comando *top*.

3) Comando PIDOF

O comando PIDOF facilita muito a identificação de um processo pelo nome do programa. Por exemplo, para saber se existe algum processo do navegador Firefox em execução:

```
# pidof Firefox
456
```

O retorno é o número do processo (*PID*) do programa em execução, no exemplo acima o Firefox.

4) Comando KILL e KILLALL

O comando KILL tem a finalidade de enviar um sinal para um processo em execução, a seguir uma lista de possíveis sinais (*SIGNAL*) que podem ser enviados:

```
# kill -l

 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
```

Os sinais permitem várias ações como interromper o processo (SIGNAL 9 e 15) ou para solicitar status de execução (SIGNAL 10). Sendo assim, o comando kill tem os seguintes objetivos:

a) Encerrar processos de sistema em execução;
b) Encerrar processos iniciados de forma acidental;
c) Encerrar processos que estejam consumindo muitos recursos do sistema;
d) Atualizar um serviço, solicitando que releia o seu arquivo de configuração (*~# kill -s 1*);
e) Solicitar o processo informações com relação ao andamento da execução de um processo (*~# kill -s 10*).

As formas de envio dos sinais podem ser pelo *PID* ou pelo nome, conforme os exemplos a seguir:

```
# kill -s <SIGNAL> <PID>
# killall -s <SIGNAL> <nome do processo>
```

5) Comando NICE

Considerando que os sistemas multiprogramados precisam estabelecer critérios para utilização de CPU, e que a prioridade do processo faz com que ele utilize mais ou menos tempo de CPU (*time-sharing*), o comando *nice* inicia um programa com mais ou para menos prioridade. Se o ajuste de prioridade para um processo for um número positivo, quer dizer que ele está sendo menos priorizado, permitindo que outros processos tenham mais tempo de CPU.

Se o ajuste for um número negativo, isso aumenta sua prioridade de execução e consome mais recursos de tempo de CPU (*time-sharing*) que outros processos.

O ajuste de prioridade possível vai do –20 (maior prioridade) até o 19 (menor prioridade). Exemplo de execução:

~# ***nice –n <prioridade> <programa>***

Executa um programa determinando sua prioridade entre -20 e 19.

6) Comando RENICE

O comando **renice** tem o mesmo objetivo do comando nice, entretanto, ele é necessário quando um programa já está em execução (e não para iniciar o programa conforme o nice). Por exemplo:

```
~# renice –n <nova prioridade> <PID>
Altera a prioridade de um processo em já execução, através do seu PID.
```

TEMA 4 – ADMINISTRAÇÃO DE GRUPOS E USUÁRIOS

A filosofia de trabalho do ambiente GNU/Linux é limitar o uso do sistema operacional para os usuários que estão operando. Com isso, o Linux possui o sistema de gerenciamento de usuários que permite criar uma conta de usuário para cada utilizador, e cada um poderá ter permissões individuais no sistema.

4.1 Tipos de usuários

Nos sistemas GNU/Linux temos usuários comuns e usuários especiais (com funções específicas), que podem ser classificados da seguinte forma:

a) root (uid = 0)
b) Usuários de sistema (uid entre 1 e 99)
c) Usuários de aplicações (uid entre 100 e 999)
d) Usuários comuns (uid >= 1000)

4.2 Arquivo de configuração de usuários (/etc/passwd)

Neste arquivo há uma entrada por linha, ou seja, um usuário por linha. As informações são definidas em campos separadas por dois-pontos (:) da seguinte forma:

```
~# man 5 passwd
/etc/passwd - User account information.
LOGIN : PASSWORD : ID : GID : NAME :   HOME : SHELL
```

Cada campo tem a função de definir os seguintes parâmetros:

LOGIN: nome de usuário/login

PASSWORD: campo de senha (obsoleto, transferido para /etc/shadow)

ID: *User IDentification* (UID), identificação única do usuário

GID: *Group IDentification*, grupo primário do usuário

NAME: nome completo e outras informações que sejam necessárias

HOME: diretório pessoal do usuário dentro de /home

SHELL: interpretador padrão para este usuário (Ex.: /bin/bash)

```
~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
apache:x:73:73:apache:/var/www:/bin/sh
aluno:x:1000:1000::/home/aluno:/bin/bash
```

4.3 Adicionando um usuário

O comando adduser permite adicionar novos usuários no sistema, utilizando o arquivo */etc/adduser.conf* para manter as opções padrão. Os usuários são classificados como usuários de sistema (criados com a opção –*system*) e usuários padrão, chamados de usuários "normais".

O usuário adicionado precisará ser identificado pelo seu identificador (*UID*), que é escolhido pelo primeiro *uid* livre no intervalo informado no arquivo *adduser.conf*. Por padrão, cada usuário em um sistema GNU / Linux recebe um grupo primário com o mesmo nome do usuário. Exemplo:

```
# adduser
Adiciona um usuário, com opção de alterar as suas
propriedades como uid, gid, shell…
```

4.4 Removendo um usuário

Exemplos para remover um usuário, com opção de remover seus arquivos:

```
~# userdel <usuario>
```

Para remover os arquivos do diretório /home:

```
~# userdel –remove <usuario>
```

4.5 Arquivo de configuração de grupos (/etc/group)

O arquivo */etc/groups* armazena as informações dos grupos e seus usuários. Cada linha do arquivo contém o grupo, sua identificação e os usuários que fazem parte deste grupo, por exemplo: *cdrom:x:30:aluno*

cdrom: nome do grupo;

senha: oferece a possibilidade de armazenar uma senha para acesso ao grupo. Esse campo fica associado com o commando *sg* (*Switch Group*);

30: número de identificação do grupo., *Group Identification (GID)*;

aluno,professor: coluna que contém os usuários do grupo, que são separados por vírgula;

Os grupos são utilizados com o objetivo de:

a) Agrupar os usuários para gerenciamento e monitoramento;
b) Facilitar o gerenciamento de permissões em arquivos e diretórios;
c) Auxiliar no compartilhamento de recursos.

4.6 Adicionando um grupo

Para adicionar um novo grupo no arquivo /etc/group é utilizado o comando addgroup, e as opções padrão de criação de grupos estão no arquivo de configuração */etc/adduser.conf*. Da mesma forma que adição de usuário, é possível adicionar um grupo para uso do sistema, com a opção *--system*.

O próximo GID utilizado é especificado para o sistema GIDs no arquivo de configuração *adduser.conf* (FIRST_SYSTEM_GID , LAST_SYSTEM_GID). Também é possível fixar o GID com a opção *--gid*.

O grupo é criado vazio, sem usuários, exemplo:

```
# addgroup <grupo>
```

4.7 Removendo um grupo

O comando *delgroup* remove grupos:

```
# delgroup <grupo>
```

TEMA 5 – POLÍTICA DE SENHAS

Uma política de senhas envolve as regras para criação de senhas, tempo de expiração e até bloqueio por inatividade. Nesta seção vamos analisar os principais parâmetros para o tratamento de senhas nos sistemas GNU/Linux, podendo ter pequenas variações nas configurações padrão em cada distribuição.

5.1 Arquivo de senhas (/etc/shadow)

No início dos sistemas Unix as senhas eram armazenadas no arquivo /etc/passwd. Com objetivo de aumentar a segurança, já que o arquivo passwd possui permissão de leitura, as senhas foram transferidas para o /etc/shadow. Além das senhas criptografadas, ele também possui informações sobre as datas de expiração e validade das contas. Esse arquivo pode ser lido somente pelo usuário root ou membros do grupo shadow.

Os campos do arquivo /etc/shadow contêm as seguintes informações:

a) Login de acesso: é o login de usuário vinculado às entradas de /etc/passwd;
b) Senha (criptografada): senha armazenada de forma criptografada. Se houver um asterisco ou ponto de exclamação, significa que a conta não possui senha, ou seja, essa conta está bloqueada. Padrão nas contas do sistema. O comando ***usermod -L*** é usado para bloquear a conta de um usuário.
c) Data da última alteração feita na senha, armazenada como o número de dias decorridos desde 01/01/1970 (*timestamp*).
d) Número de dias até que seja solicitado troca de senha
e) Número de dias que devem se passar até que seja possível alterar a senha.
f) Período de aviso, número de dias antes que uma alteração seja necessária.
g) Número de dias após a última alteração de senha.
h) Número de dias de avisos antes da expiração da senha.
i) Dias entre expiração e desativação.
j) Número de dias que decorrerão entre a expiração da senha e o travamento da conta do usuário (l).
k) Data de expiração do login.
l) Data final na qual a conta será desabilitada.
m) Flag especial (este campo se encontra vazio).

5.2 Arquivo de definição de padrão de senhas

As definições de parâmetros padrão das contas de usuário estão no arquivo /etc/login.defs, as configurações principais estão nas opções a seguir:

- **PASS_MIN_LEN:** comprimento mínimo de caracteres na definição de senha;
- **PASS_MAX_LEN:** comprimento máximo de caracteres na definição de senha;
- **PASS_MAX_DAYS:** número máximo de dias que uma senha pode ser usada;
- **PASS_MIN_DAYS:**número mínimo de dias antes do aviso de troca;
- **PASS_WAR_AGE:** dias para senha expirar após a notificação de troca;
- **LOGIN_RETRIES:** máximo de tentativas com erros de senhas no login;
- **LOGIN_TIMEOUT:** tempo máximo em segundos para o login.

5.3 Alterando política de expiração de senhas e usuários

O comando ***chage*** altera as datas de expiração das contas de usuário. As opções utilizadas são:

-E data: determina a data de expiração (utilizar o formato ano, mês e dia AAAA-MM-DD). A data de validade da conta pode ser removida utilizando o valor "-1";

-l: exibe as configurações de uma conta do usuário. Qualquer usuário pode exibir as suas configurações e expiração de senha e conta;

-**d *AAAA-MM-DD***: determina o número N de dias da última alteração de senha desde 01/01/1970 (aceita o formato AAAA-MM-DD);

-E *AAAA-MM-DD*: determina o número de dias em que a conta será desabilitada desde 01/01/1970 (aceita o formato AAAA-MM-DD);

-I: Determina o número de dias de inatividade após a expirar a senha da conta que será desabilitada;

-m: determina o número mínimo de dias de validade de uma senha. Um valor zero permite que o usuário troque de senha a qualquer hora;

-M: determina o número máximo de dias de validade de uma senha, fazendo com que o usuário a troque;

-W: determina o número de dias em que um aviso será exibido ao usuário antes da troca da senha.

Exemplos:

1. Desabilitar (expirar) a conta do usuário aluno em 31/12/2023:

```
a.# chage -E 2023-12-31 aluno
```

2. Apenas informando as datas da conta do usuário (sem nenhuma opção):

```
# chage aluno
Changing the aging information for aluno
Enter the new value, or press ENTER for the default
Minimum Password Age [0]: 10
Maximum Password Age [99999]: 60
Last Password Change (YYYY-MM-DD) [2023-10-17]:
Password Expiration Warning [7]: 7
Password Inactive [-1]: 10
Account Expiration Date (YYYY-MM-DD) [2023-12-31]:
2023-12-31
```

FINALIZANDO

Com objetivo de fixar conhecimento sobre os principais termos utilizados nesta etapa, vamos listar a seguir os principais conceitos revisados:

- ***Background*:** no contexto utilizado nesta etapa, *background* se refere aos processos que são executados em segundo plano. Os processos em *background* não necessitam desta interação com o usuário, portanto, não estão necessariamente vinculados a uma sessão ativa ou terminal;
- **BIOS (*Basic Input/Output System*):** sistema básico de Entrada e Saída, é um pequeno programa gravado em um chip na placa do sistema (*EEPROM* ou *FLASHROOM*). Quando inicia um computador, o BIOS é o primeiro software a ser executado;
- ***Daemon*:** um *daemon* é um tipo de programa que funciona discretamente em segundo plano, em vez de o usuário controlar diretamente, ele fica aguardando para que seja ativado pela ocorrência de um evento ou condição específica. Os serviços do sistema operacional são iniciados desta forma e são chamados à medida que são utilizados. Por exemplo: *Spool* de Impressão, Servidor Gráfico (*Xorg*), entre outros.
- ***Foreground*:** no contexto utilizado nesta etapa, *foreground* se refere aos processos que são executados em primeiro plano. Os processos executados em *foreground* são aqueles que necessitam de interação direta com o usuário;

- **Gerenciador de inicialização:** é um software especial responsável por inicializar o sistema de maneira adequada, sendo responsável por reconhecer as partições dos discos e iniciar o *Kernel* do sistema operacional;
- **GID (*Group IDentification*):** identificador de grupo, os usuários são organizados em grupos e os arquivos (e diretórios) também. O GID de um arquivo é inicialmente herdado do usuário que cria o arquivo.
- ***HOME*:** no diretório/home ficam os arquivos pessoais, como documentos e imagens, sempre dentro de pastas que levam o nome de cada usuário. O gerenciador de usuários utiliza a diretiva *HOME*, no arquivo */etc/password*, para definir qual é o diretório do usuário;
- **MBR (*Master Boot Record*):** é um padrão que indica a forma como os dados serão armazenados no disco. MBR é uma sigla para "*Master Boot Record*", consiste nos primeiros 512 Bytes de um disco, também chamado de trilha 0 ou setor de boot. Está sendo substituído pelo GPT (*GUID Partition Table*), um padrão mais novo e com menos limitações;
- **POST (*Power On Self Test*):** também pode ser chamada de autoteste de inicialização, ou seja, uma rotina que ocorre na inicialização de um computador com objetivo de realizar uma sequência de testes para verificar se o sistema se encontra em estado operacional, ou seja, com condições de uso;
- **Processo:** é um programa em execução. Os processos são identificados pelos seus identificadores de processos, números inteiros (*PID*). É função do sistema operacional iniciar e controlar a execução dos processos do usuário e do sistema;
- **root:** o root é o usuário com maior nível de autorização dentro do Linux. Por exemplo, o root pode criar usuários, remover usuários, alterar a configuração do sistema, entre outras funções;
- ***Runlevel* (nível de execução):** os runlevels servem basicamente para especificar como o sistema deve se comportar durante a inicialização, execução e desligamento;
- ***Shadow*:** no contexto de gerenciamento de usuários, Shadow é utilizado para conferir segurança no armazenamento das senhas. A criptografia e as permissões do arquivo de senhas (*/etc/shadow*) caracterizam as senhas de sombra;

- **Signal (Sinal):** são interrupções de software implementadas pelo sistema operacional para permitir que eventos sejam disparados quando essas interrupções acontecerem;
- ***Systemd*:** sistema que gerencia quase toda a inicialização do sistema GNU/Linux, padrão em diversas distribuições, que também consegue gerenciar todo ciclo de vida de um serviço (*Start/Stop/Status*);

REFERÊNCIAS

DEITEL, P. J.; DEITEL, H. M.; CHOFFNES, D. R. **Sistemas Operacionais**. 3. ed. Pearson, 2017. (BVP)

HILL, B. M.; BACON, J. **O livro oficial do ubuntu**. 2. ed. Porto Alegre: Bookman, 2008. ISBN 9788577803286. (BVMB)

LINUX.ORG. Disponível em: <https://www.linux.org/>. Acesso em: 7 dez. 2023.

MACHADO, F. B.; MAIA, P. L. **Fundamentos de Sistemas Operacionais**. Rio de Janeiro: LTC, 2011. (BVMB)

NEMETH, E.; SNYDER, G.; HEIN, T. R. **Manual Completo do Linux:** guia do administrador. 2. ed. São Paulo: Pearson Prentice Hall, 2007. (BVP)

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015. (BVMB)

TANENBAUM, A. S. **Sistemas operacionais modernos**. 4. ed. São Paulo: Pearson Prentice Hall, 2008.